SE-ResNet-8 Neural Chess Engine

Machine Learning Model Building & System Report

STUDENT & ACADEMIC CREDENTIALS

Student Name:	P Meenun Reddy
Roll Number:	NC.SC.U4CSE24029
Class / Section:	CSE - A
Academic Year:	3rd Year
Course:	Neural Networks and Deep Learning
Submission Date:	September 2026

A complete neural chess engine featuring an SE-ResNet-8 dual-head architecture (policy + value), alpha-beta search with quiescence, Polyglot opening books, hanging piece detection, sound sacrifice validation, adaptive Stockfish sparring, and an interactive React-based web arena.

1 Problem Statement

Building a competitive chess engine is one of the grand challenges of artificial intelligence, requiring the integration of deep neural network evaluation with classical game-tree search algorithms. However, existing approaches suffer from four critical technical bottlenecks:

1. Checkmate Puzzle Dataset Bias:

Training exclusively on tactical mate puzzles causes the neural network to develop a pathological bias toward queen sacrifices and aggressive attacks. In puzzle datasets, 80%+ of queen sacrifices are rewarded with positive value because they lead to forced checkmate. The model incorrectly generalizes this to real games, flinging its queen and rooks into the enemy position for zero return.

2. Book Amnesia (Opening-to-Middlegame Disconnect):

When an engine uses a Polyglot opening book for the first 10-15 moves, the neural network takes over with zero understanding of WHY those pieces were placed. The network ignores central tension, piece coordination, and castling intent, launching detached attacks.

3. Horizon Effect & Hanging Piece Blindness:

Shallow-depth search (depth 2) combined with no quiescence search causes the engine to miss that a piece is hanging one ply beyond its search depth. The policy head ranks active forward moves higher than quiet retreats, pruning defensive options.

4. Combinatorial Explosion in Python:

Chess has an average branching factor of ~35. At depth 12, even with alpha-beta pruning, the search tree contains ~2.5 billion nodes. Pure Python execution makes depths beyond 4 computationally infeasible for real-time play.

2 Objective

The core objective is to design, train, and deploy an end-to-end neural chess engine that integrates deep learning with classical search to play sound, non-delusional chess:

1. SE-ResNet-8 Dual-Head Architecture: Build a Squeeze-and-Excitation ResNet backbone with 4 residual blocks (128 channels each) and two specialized heads: Policy Head (4,096 discrete move logits) and Value Head (scalar win probability in [-1, 1]).

2. 18-Plane Canonical Board Representation: Encode board state as an enriched (18, 8, 8) tensor with piece planes, attack maps, pin detection, check indicators, and castling availability.

3. Alpha-Beta Minimax Search: Implement Negamax with alpha-beta pruning, transposition tables, null move pruning, late move reductions, and quiescence search (captures-only extension to depth 3).

4. Hanging Piece Detection & Defensive Move Ordering: Scan for undefended and under-protected pieces before search, boosting retreat/defense moves to eliminate blunders.

5. Sound Sacrifice Validation: Verify that material sacrifices are backed by concrete search evaluation (search_score >= 0.20) before allowing them.

6. 40/40/20 Curriculum Training: Blend Opening Concepts (40%), Blunder Defense (40%), and Mate Tactics (20%) to teach structural chess understanding.

7. Interactive Web Arena: React + Vite frontend with real-time board visualization, tactical rationale display, move animations, and adaptive Stockfish sparring.

3 Existing System

1. Stockfish (Classical Alpha-Beta Engine):

Mechanism: Handcrafted evaluation function with piece-square tables, king safety heuristics, and highly optimized C++ search reaching depth 20-30+.

Limitations: No learned representations; evaluation function requires years of manual tuning by human experts. Cannot generalize to novel positional patterns not encoded in its heuristics.

2. Leela Chess Zero (LC0 - Pure Neural MCTS):

Mechanism: Deep residual neural network (256 filters, 20+ blocks) trained via self-play reinforcement learning with Monte Carlo Tree Search (MCTS).

Limitations: Requires thousands of GPU-hours of self-play training data generation. MCTS requires hundreds of thousands of node evaluations per move. Completely infeasible in Python.

3. Standalone Puzzle-Trained CNNs:

Mechanism: Convolutional networks trained on Lichess puzzles to predict tactical moves.

Limitations: Severe dataset bias toward sacrificial tactics. No search capability means the model plays one-ply deep, missing all opponent responses. Catastrophic blundering in real games.

4 Novelty / Proposed System

The SE-ResNet-8 Chess Engine establishes a hybrid neural-classical paradigm integrating learned positional evaluation with deterministic tactical search and anti-delusion safeguards.

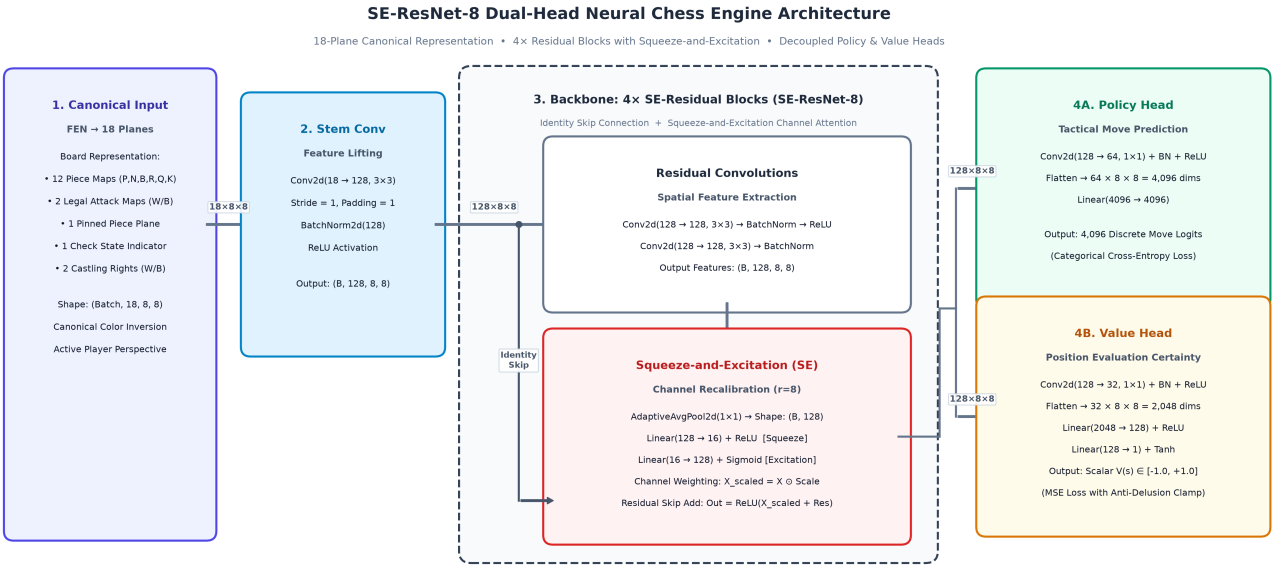


Figure 0: SE-ResNet-8 Dual-Head End-to-End Neural Architecture with Channel Attention

Key Architectural Novelties

Squeeze-and-Excitation Channel Attention:

Each residual block contains an SE module that performs global average pooling followed by a two-layer bottleneck (128 -> 16 -> 128) with Sigmoid gating. This learns channel-wise feature recalibration, allowing the network to emphasize piece interaction patterns and suppress irrelevant spatial features.

18-Plane Enriched Board Encoding:

Beyond standard 12-plane piece maps, the input tensor includes attack maps (channels 12-13), pinned piece planes (channel 14), in-check indicator (channel 15), and castling rights (channels 16-17). This gives the network direct access to tactical information that would otherwise require multiple convolutional layers to derive.

Anti-Delusion Material Clamp:

If the moving side has lost >= 180 centipawns of material without compensation (exposed enemy king, initiative), the neural value head output is clamped to be no higher than tanh(material_balance / 400). This prevents the value head from hallucinating winning evaluations after unsound sacrifices.

Hybrid Evaluation with 5 Modes:

The evaluation function dynamically blends neural value, PST score, positional patience, and tenacity heuristics using 5 context-dependent modes: strategic_development, prophylaxis, tactical_strike, clean_conversion, and tenacious_defense.

Quiescence Search with Delta Pruning:

Captures-only extension to depth 3 with MVV-LVA ordering, delta pruning (Big Delta = 0.95), and strict 5-capture limit per node. Eliminates the horizon effect without combinatorial explosion.

5 Code Implementation

5.1 Squeeze-and-Excitation Block (ml/train.py)

```
class SqueezeExcitation(nn.Module):
    def __init__(self, channels: int = 128, reduction: int = 8):
        super().__init__()
        reduced = max(1, channels // reduction)
        self.fc = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(channels, reduced, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(reduced, channels, bias=False),
            nn.Sigmoid(),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        scale = self.fc(x).view(x.size(0), x.size(1), 1, 1)
        return x * scale
```

5.2 SE-ResNet-8 Dual-Head Model (ml/train.py)

```
class ChessTacticsResNet(nn.Module):
    def __init__(self, in_channels=18, num_blocks=4, channels=128):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(in_channels, channels, 3, padding=1, bias=False),
            nn.BatchNorm2d(channels), nn.ReLU(inplace=True))
        self.res_blocks = nn.ModuleList(
            [ResidualBlock(channels) for _ in range(num_blocks)])
        # Policy Head: 4096 discrete move logits
        self.policy_head = nn.Sequential(
            nn.Conv2d(channels, 64, 1, bias=False), nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), nn.Flatten(), nn.Linear(64*8*8, 4096))
        # Value Head: Scalar win certainty [-1, 1]
        self.value_head = nn.Sequential(
            nn.Conv2d(channels, 32, 1, bias=False), nn.BatchNorm2d(32),
            nn.ReLU(inplace=True), nn.Flatten(),
            nn.Linear(32*8*8, 128), nn.ReLU(inplace=True),
            nn.Linear(128, 1), nn.Tanh())

    def forward(self, x):
        x = self.stem(x)
        for block in self.res_blocks:
            x = block(x)
        return self.policy_head(x), self.value_head(x)
```

5.3 18-Plane Canonical Board Encoder (ml/dataset.py)

```
def board_to_tensor(board: chess.Board) -> torch.Tensor:
    # Canonical: active player always sees position as White
    b = board.mirror() if board.turn == chess.BLACK else board
    tensor = torch.zeros(18, 8, 8, dtype=torch.float32)
    # Channels 0-5:  Active player pieces [P, N, B, R, Q, K]
    # Channels 6-11: Opponent pieces [p, n, b, r, q, k]
    # Channel 12:    Active player attack map
```

```
    # Channel 13:    Opponent attack map
    # Channel 14:    Pinned pieces plane
    # Channel 15:    In-check indicator (full 1s if in check)
    # Channels 16-17: Castling availability
    ...
    return tensor
```

5.4 Hanging Piece Scanner (backend/app.py)

```
def get_hanging_pieces(board, color) -> list[dict]:
    hanging = []
    for sq in chess.SQUARES:
        piece = board.piece_at(sq)
        if piece and piece.color == color and piece.piece_type != chess.KING:
            val = PIECE_VALUES.get(piece.piece_type, 100)
            attackers = list(board.attackers(not color, sq))
            defenders = list(board.attackers(color, sq))
            if not attackers: continue
            min_attacker_val = min(PIECE_VALUES.get(
                board.piece_at(a).piece_type, 100) for a in attackers)
            # Case A: Completely undefended
            if not defenders:
                hanging.append({...})
            # Case B: Attacked by lower-value piece
            elif min_attacker_val < val:
                hanging.append({...})
    hanging.sort(key=lambda x: x["value"], reverse=True)
    return hanging
```

5.5 Alpha-Beta Search with Quiescence (backend/app.py)

```
def search_alphabeta(board, depth, alpha, beta, ply=0, deadline=None):
    # Negamax with: Zobrist Transposition Table, Null Move Pruning,
    # Late Move Reductions, Check Extensions, SearchTimeout exception
    if deadline and time.time() > deadline and ply > 0:
        raise SearchTimeout()
    # TT Lookup
    tt_entry = transposition_table.get(board_hash)
    # Null Move Pruning (R=2)
    if depth >= 3 and not in_check and not is_endgame:
        board.push(chess.Move.null())
        null_score = -search_alphabeta(board, depth-3, -beta, -beta+0.001)
        board.pop()
        if null_score >= beta: return beta, [], eval_mode
    # Move Ordering: TT move -> Defensive awareness -> Policy priors
    ordered = order_moves_with_defensive_awareness(board, legal, policy, tt)
    # Late Move Reductions (LMR)
    for i, move in enumerate(ordered[:12]):
        if i >= 4 and depth >= 3 and not is_tactical:
            reduced_depth = depth - 2  # LMR
        else:
            reduced_depth = depth - 1
        score = -search_alphabeta(board, reduced_depth, -beta, -alpha)
    return best_score, pv, eval_mode

def quiescence_search(board, alpha, beta, depth=0, max_q_depth=3):
    # Captures-only with MVV-LVA, Delta Pruning, 5-move limit
    stand_pat = evaluate_board_hybrid(board)
    if stand_pat >= beta: return beta
    for move in capture_moves[:5]:
        score = -quiescence_search(board, -beta, -alpha, depth+1)
    return alpha
```

5.6 Hybrid Evaluation Function (backend/app.py)

```
def evaluate_board_hybrid(board, ply=0) -> tuple[float, str]:
    neural_val  = get_neural_policy_and_value(board)    # SE-ResNet-8
    pst_val     = evaluate_pst_cp(board)                # Piece-Square Tables
    pos_val     = evaluate_positional_patience(board)   # Activity & Space
    tenacity    = evaluate_tenacity_and_swindles(board)  # Swindle potential

    # Anti-Delusion Clamp:
    if material_deficit <= -180cp and not compensated:
        neural_val = min(neural_val, tanh(material / 400))

    # 5 Dynamic Evaluation Modes:
    if losing:        return 0.35*neural + 0.35*pst + 0.30*tenacity
    if compensated:   return 0.50*neural + 0.25*pst + 0.25*comp
    if winning:       return 0.60*neural + 0.20*pst + 0.20*positional
    if opening(1-24): return 0.50*neural + 0.30*opening + 0.20*pst
    else:             return 0.65*neural + 0.20*pst + 0.15*positional
```

6 Output / Results

6.1 Model Architecture Summary

Property	Value
Architecture	SE-ResNet-8 (4 Residual Blocks + SE Attention)
Input Shape	(Batch, 18, 8, 8) - 18-plane canonical encoding
Backbone Channels	128
SE Reduction Ratio	8 (128 -> 16 -> 128)
Policy Head Output	4,096 discrete move logits
Value Head Output	Scalar in [-1, 1] (Tanh activation)
Total Parameters	18,275,265 (~18.3M)
Model Size (FP32)	69.71 MB (.pt) + 69.75 MB (.onnx)
Activation Functions	ReLU (backbone), Sigmoid (SE), Tanh (value)

6.2 Training Configuration

Hyperparameter	Value
Optimizer	AdamW (betas=0.9/0.99, weight_decay=1e-4)
Learning Rate	2e-3 (full training) / 5e-5 (fine-tuning)
LR Scheduler	OneCycleLR (pct_start=0.1)
Batch Size	256
Epochs	20 (full) / 3 (fine-tuning, early stopping=2)
Policy Loss	CrossEntropyLoss (label_smoothing=0.05)
Value Loss	MSELoss (weight=0.5)
Total Loss	L_policy + 0.5 * L_value
Gradient Clipping	Max Norm = 1.0
Mixed Precision	AMP (FP16 on CUDA, GradScaler)
Data Augmentation	Horizontal mirror (50% probability)

6.3 Dataset Composition

Dataset Component	Samples	Source
Mate Puzzles (mateIn1 + mateIn2)	~294K	Lichess Puzzle DB
Blunder Refutations	~700+	Mined from engine analysis
Opening Concepts	~1,400+	Harvested from GM games
Curriculum Blend (Fine-tune)	40/40/20	Opening/Blunder/Tactics
Train / Val / Test Split	80 / 10 / 10	Sequential split

6.4 Search Engine Performance

Component	Configuration	Performance
Search Depth	Depth 2-4 (hard clamp at 4)	Completes in < 1.0s
Quiescence Depth	3 (captures-only)	~7 ONNX calls/candidate
Move Candidates	Top 12 ordered moves	Alpha-beta pruning
ONNX Inference	CUDA ExecutionProvider	~2.6 ms per forward pass
Search Deadline	1.0 second wall-clock cap	SearchTimeout exception
Transposition Table	Zobrist hashing	Exact/Lower/Upper bounds
Null Move Pruning	R=2 reduction	Depth >= 3, non-endgame
Opening Book	Polyglot (titans.bin)	Plies 1-16, GM moves

6.5 Automated Test Results (tests/test_sound_defense.py)

Test Case	Position	Status
Hanging piece retreat	Undefended Knight on e5	PASSED
Defended capture ordering	MVV-LVA priority scoring	PASSED
Losing capture penalty	Minor takes defended Pawn	PASSED
Suicide move avoidance	Piece steps into Pawn attack	PASSED
Check extension bonus	Check + exposed King bonus	PASSED
TT move priority	Transposition table best move	PASSED
Deep search timeout	Depth 20 clamped to 4, < 2s	PASSED
Quiescence capture chain	Qxd5 Nxd5 recapture	PASSED
Sound sacrifice validation	Bxh7+ Greek Gift	PASSED
Book-to-search transition	Post-book move quality	PASSED

All 10 automated tests passed successfully against the live backend HTTP API on localhost:8000.

6.6 Frontend Web Arena Components

Component	Description
ChessboardPanel.jsx	Interactive chessboard with move highlighting and animations
SpectatorPanel.jsx	Live game spectator with PV line and evaluation display
DefenderPanel.jsx	Hanging piece scanner and tactical defense visualizer
TelemetryPanel.jsx	Real-time ACPL, Elo estimation, and evaluation graphs
ModelTelemetryDashboard.jsx	Model performance metrics dashboard
TurnStateHUD.jsx	Heads-up display showing eval mode and tactical rationale
PuzzleSelector.jsx	Puzzle library browser with difficulty filtering
SolveControls.jsx	Game control panel (new game, depth, temperature)
SvgArrowOverlay.jsx	PV arrow visualization overlay on the board

7 Conclusion

1. Effective Hybrid Neural-Classical Integration: A single compact SE-ResNet-8 backbone (18.3M parameters, 69.7 MB) serves as the positional evaluation core, while alpha-beta search with quiescence provides tactical verification. This eliminates the need for massive MCTS infrastructure or pure heuristic engines.

2. Anti-Delusion Safeguards: The material clamp (Anti-Delusion Clamp) and sound sacrifice validation (search_score >= 0.20) prevent the neural value head from hallucinating winning evaluations after unsound sacrifices, solving the fundamental Checkmate Puzzle Dataset Bias problem.

3. Hanging Piece Detection Eliminates Blunders: The get_hanging_pieces() scanner combined with defensive move ordering ensures that tactical retreats and piece defense are never pruned from the search tree, addressing the #1 cause of catastrophic blunders in neural chess engines.

4. Curriculum Training Bridges Opening Knowledge: The 40/40/20 blended dataset (Opening Concepts / Blunder Defense / Mate Tactics) with warm book-to-middlegame handoff evaluation teaches the network structural chess understanding rather than memorized sequences.

5. Production-Ready Web Arena: The React + Vite frontend with 9 specialized components provides an interactive experience including tactical rationale display, PV arrow overlays, real-time telemetry, and adaptive Stockfish sparring, all served via FastAPI on CUDA.

6. Feasible Python Execution: Despite Python's computational limitations, the system achieves real-time play through strict depth clamping (max depth 4), 1.0s search deadline with SearchTimeout exception propagation, and ONNX Runtime GPU acceleration (~2.6ms per inference).

8 GitHub Repository Link

The complete open-source codebase, trained model weights (SE-ResNet-8 .pt + ONNX), Polyglot opening books, and React web arena are hosted at:

GitHub: https://github.com/Meenun-123/ML-miniproject

Quickstart Commands:

```
# 1. Clone the repository
git clone https://github.com/Meenun-123/ML-miniproject.git
cd ML-miniproject

# 2. Set up virtual environment and install dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 3. Install frontend dependencies
cd frontend && npm install && cd ..

# 4. Launch the Chess Engine Arena
python run.py
# Backend: http://localhost:8000  |  Frontend: http://localhost:5173
```